Polytech Lyon

Optimisation Discrète

Projet : Vehicle Routing Problem with Time Windows

Rapport de projet

Étudiants :	El Bouazzaoui Ali
	Nadir Ayoub
Enseignant :	Stéphane Bonnevay
Date :	4 mai 2026

Ce rapport présente notre travail sur la résolution du problème de tournées de véhicules avec fenêtres de temps (VRPTW) à l'aide de métaheuristiques. Le sujet demande de modéliser le problème, de structurer le code, de déterminer le nombre minimal de véhicules, de générer des solutions aléatoires, puis d'implémenter et comparer deux métaheuristiques.

Contents

1 Introduction **4**

2 Mode opératoire **4**

2.1 Prérequis 4

2.2 Organisation des fichiers 4

2.3 Mode aperçu (défaut) 5

2.4 Mode expérimentation 5

2.5 Mode bonus — résolution exacte 6

2.6 Récapitulatif des options de `main.py` 6

3 Organisation du projet **6**

3.1 Phase 1 : Fondations du projet 6

3.2 Phase 2 : Détermination du nombre minimal de véhicules 6

3.3 Phase 3 : Génération de solutions initiales 6

3.4 Phase 4 : Première métaheuristique 7

3.5 Phase 5 : Deuxième métaheuristique 7

3.6 Phase 6 : Protocole expérimental et comparaison 7

4 Modélisation du problème **7**

4.1 Description générale 7

4.2 Ensembles 7

4.3 Paramètres 8

4.4 Variables de décision 8

4.5 Fonction objectif 8

4.6 Contraintes principales 8

4.6.1 Visite unique de chaque client 8

4.6.2 Conservation du flot 9

4.6.3 Capacité 9

4.6.4 Fenêtres de temps 9

4.7 Remarque sur l'utilisation d'OR-Tools 9

5 Structure du code **9**

5.1 Évolution de l'architecture 10

5.2 Justification des choix de fichiers 10

5.3 Pourquoi cette structure nous a semblé adaptée 11

5.4 Lien entre structure logicielle et démarche scientifique 11

5.5 `main.py` 12
5.6 `model.py` 12
5.7 `io_utils.py` 12
5.8 `evaluate.py` 12
5.9 `construction.py` 13
5.10 `neighbors.py` 13
5.11 `solver.py` 13
5.12 `meta/simulated_annealing.py` 13
5.13 `meta/tabu_search.py` 14
5.14 `bonus_exact.py` 14

6 Détermination du nombre minimal de véhicules 14
6.1 Principe général 14
6.2 Borne inférieure théorique 14
6.3 Choix méthodologique 15
6.4 Heuristique utilisée pour tester une valeur de k 15
6.5 Critère de faisabilité 16
6.6 Remarque 16
6.7 Résultats obtenus sans fenêtres de temps 16
6.8 Influence des fenêtres de temps sur la borne inférieure 17
6.9 Première intégration effective des fenêtres de temps 18
6.10 Premiers résultats VRPTW 18
6.11 Limites observées à ce stade 19
6.12 Première extension des métaheuristiques au cas VRPTW 19
6.13 Premier essai expérimental VRPTW 19
6.14 Campagne VRPTW en mode rapide 20
6.15 Analyse de cette première campagne VRPTW 20
6.16 Campagne VRPTW en mode long 21
6.17 Analyse de la campagne VRPTW longue 21

7 Génération d'une solution initiale 22
7.1 Choix retenu 22
7.2 Cas sans fenêtres de temps 22
7.3 Cas avec fenêtres de temps 23
7.4 Intérêt de cette construction 23
7.5 Rôle de cette solution 23

8 Choix des méthodes de résolution 23

8.1 Voisinages envisagés . . . 23
8.2 Première implémentation des voisinages . . . 24
8.3 Première implémentation du recuit simulé . . . 24
8.4 Pseudo-code simplifié du recuit simulé . . . 25
8.5 Résultats préliminaires . . . 25
8.6 Première implémentation de la recherche tabou . . . 25
8.7 Observation importante sur le coût de calcul . . . 25
8.8 Résultats préliminaires de la recherche tabou . . . 26
8.9 Mise en place du protocole expérimental . . . 26
8.10 Paramètres retenus . . . 26
8.11 Critères de comparaison retenus . . . 26
8.12 Répétitions et reproductibilité . . . 27
8.13 Premiers résultats comparatifs . . . 27
8.14 Indicateurs de dispersion . . . 28
8.15 Premier essai en mode long . . . 28
8.16 Courbe de convergence . . . 29
8.17 Courbe de convergence en VRPTW . . . 29
8.18 Extension du test long à `data102.vrp` . . . 30
8.19 Résultats sur `data1101.vrp` et `data1102.vrp` . . . 30
8.20 Résultats sur `data111.vrp` et `data112.vrp` . . . 31
8.21 Résultats sur `data1201.vrp` et `data1202.vrp` . . . 31
8.22 Résultats sur `data201.vrp` et `data202.vrp` . . . 31
8.23 Synthèse de la première campagne longue . . . 32
8.24 Lecture synthétique du corpus et cas marquants . . . 33
8.25 Exemple de visualisation de tournées . . . 34

9 Format des données **34**

10 Bonus : résolution exacte sur sous-instances **35**

11 Limites et perspectives **36**

12 Conclusion **36**

12.1 Comment nous cherchons l'optimum . . . 36
12.2 Améliorations envisageables . . . 37
12.3 Critique du protocole expérimental . . . 37

1. Introduction

Le projet étudié porte sur le **Vehicle Routing Problem with Time Windows (VRPTW)**. L'objectif est de déterminer un ensemble de tournées partant et revenant au dépôt, de manière à desservir chaque client exactement une fois, en respectant les contraintes du problème et en minimisant la distance totale parcourue.

Le sujet demande explicitement :

- de modéliser le problème et de structurer le code ;
- de déterminer, pour chaque jeu de données, le nombre minimum de véhicules ;
- de créer un générateur aléatoire de solutions ;
- d'implémenter deux métaheuristiques ;
- de comparer les résultats obtenus ;
- de commencer sans tenir compte des fenêtres de temps, puis de les ajouter ensuite.

Nous considérons d'abord le problème **sans fenêtres temporelles**, ce qui revient à traiter un **Capacitated Vehicle Routing Problem (CVRP)**. Les fenêtres de temps sont ensuite réintroduites pour traiter le problème complet.

2. Mode opératoire

Cette section décrit comment reproduire nos expériences ou tester les algorithmes sur de nouvelles instances.

2.1. Prérequis

Le projet nécessite **Python 3.10** ou supérieur. Les dépendances pour l'affichage des figures sont installables avec :

```
pip install matplotlib numpy
```

La résolution exacte (section bonus) nécessite en plus :

```
pip install ortools
```

2.2. Organisation des fichiers

La structure du projet est la suivante :

```
OptimisationAli-Ayoub/
  data/                # instances .vrp a placer ici
    data101.vrp
    data102.vrp
    data111.vrp
    data112.vrp
    data201.vrp
    data202.vrp
    data1101.vrp
```

```
    data1102.vrp
    data1201.vrp
    data1202.vrp
  src/
    main.py              # point d'entree principal
    model.py             # structures de donnees
    io_utils.py          # lecture des fichiers .vrp
    evaluate.py          # calcul distances et faisabilite
    construction.py      # construction de solutions initiales
    neighbors.py         # operateurs de voisinage
    solver.py            # pilotage des experiences
    bonus_exact.py       # resolution exacte (OR-Tools)
    meta/
      simulated_annealing.py
      tabu_search.py
  rapport.pdf
```

Les fichiers d'instances `.vrp` doivent être placés dans un dossier `data/` à la racine du projet avant de lancer les commandes.

Toutes les commandes ci-dessous sont à lancer **depuis la racine du projet** (dossier `OptimisationAli-Ayoub/`)

2.3. Mode aperçu (défaut)

La commande suivante affiche un résumé complet des instances, le nombre minimal de véhicules en CVRP et VRPTW, des solutions aléatoires générées, et une première comparaison rapide des deux métaheuristiques :

```
python src/main.py
```

2.4. Mode expérimentation

CVRP — preset rapide. Lance une campagne sur l'ensemble des instances sans fenêtres de temps, en quelques secondes :

```
python src/main.py --mode experiment --preset quick
```

CVRP — preset long. Lance la campagne complète avec davantage d'itérations et trois répétitions par instance (quelques minutes) :

```
python src/main.py --mode experiment --preset long
```

VRPTW. L'option `-time-windows` active les fenêtres de temps :

```
python src/main.py --mode experiment --preset long --time-windows
```

Instance unique avec détail run par run.

```
python src/main.py --mode experiment --preset quick \
    --instances data101.vrp --details
```

Les résultats sont enregistrés automatiquement dans `experiment_results.log`. Un autre fichier peut être choisi avec `-output-file nom.log`.

2.5. Mode bonus — résolution exacte

Ce mode requiert OR-Tools. Il lance une résolution exacte sur des sous-instances de taille croissante extraites de `data101.vrp` :

```
python src/main.py --mode bonus \
    --sizes 5 10 15 20 25 30 --time-limit 10
```

2.6. Récapitulatif des options de `main.py`

Option	Description
`-mode`	`overview` (défaut), `experiment`, `bonus`
`-preset`	`quick` ou `long`
`-instances`	fichiers `.vrp`, ex. `data101.vrp data202.vrp`
`-limit`	nombre maximal d'instances à traiter
`-time-windows`	active les contraintes temporelles (VRPTW)
`-method`	`both` (défaut), `sa` ou `tabu`
`-details`	affiche le détail par répétition
`-output-file`	fichier de sortie pour les résultats
`-sizes`	tailles des sous-instances (mode bonus)
`-time-limit`	limite de temps par sous-instance (mode bonus, en s)

3. Organisation du projet

Afin de répondre progressivement aux différentes questions du sujet, nous avons découpé le travail en plusieurs phases.

3.1. Phase 1 : Fondations du projet

Cette phase regroupe :

- la modélisation mathématique du problème ;
- la définition de la structure du code.

3.2. Phase 2 : Détermination du nombre minimal de véhicules

Cette phase est décomposée en deux sous-parties :

- sans fenêtres de temps ;
- avec fenêtres de temps.

3.3. Phase 3 : Génération de solutions initiales

La méthode retenue pour satisfaire la consigne de génération aléatoire consiste à construire des **solutions aléatoires faisables**.

3.4. Phase 4 : Première métaheuristique

Le **recuit simulé** est retenu comme première métaheuristique. Une première version fonctionnelle permet de valider la chaîne complète :

- génération d'une solution initiale ;
- génération d'un voisin faisable ;
- acceptation déterministe ou probabiliste ;
- mémorisation de la meilleure solution rencontrée.

3.5. Phase 5 : Deuxième métaheuristique

La **recherche tabou** est retenue comme seconde métaheuristique. Une première version permet de disposer rapidement des deux approches demandées par le sujet avant de lancer les comparaisons expérimentales détaillées.

3.6. Phase 6 : Protocole expérimental et comparaison

L'infrastructure expérimentale distingue clairement deux usages :

- le préréglage technique `quick`, utilisé comme «mode rapide » pour valider les implémentations et obtenir un premier ordre de grandeur ;
- le préréglage technique `long`, utilisé comme «mode long » pour les comparaisons plus rigoureuses, avec davantage d'itérations et plusieurs répétitions.

4. Modélisation du problème

4.1. Description générale

On considère un dépôt unique et un ensemble de clients. Chaque client possède :

- des coordonnées cartésiennes ;
- une demande ;
- éventuellement une fenêtre de temps ;
- un temps de service.

Les véhicules sont supposés identiques et possèdent tous la même capacité maximale Q.

L'objectif principal est de minimiser la distance totale parcourue. Le nombre de véhicules n'étant pas fixé dans le sujet, il doit être déterminé.

4.2. Ensembles

$$V = \{0, 1, \ldots, n\}$$

désigne l'ensemble des sommets, où :

- 0 représente le dépôt ;
- $C = \{1, \ldots, n\}$ représente l'ensemble des clients.

4.3. Paramètres

- d_{ij} : distance entre les sommets i et j ;
- q_i : demande du client i ;
- Q : capacité maximale d'un véhicule ;
- a_i : date de début de la fenêtre de temps du client i ;
- b_i : date de fin de la fenêtre de temps du client i ;
- s_i : durée de service chez le client i.

4.4. Variables de décision

Une modélisation classique consiste à utiliser des variables binaires :

$$x_{ij} = \begin{cases} 1 & \text{si un véhicule va directement de } i \text{ vers } j, \\ 0 & \text{sinon.} \end{cases}$$

avec

$$x_{ij} \in \{0, 1\} \qquad \forall i, j \in V,\ i \neq j.$$

Pour la version avec fenêtres de temps, on introduit également :

$$t_i = \text{instant de début de service au client } i.$$

avec

$$t_i \geq 0 \qquad \forall i \in V.$$

4.5. Fonction objectif

L'objectif est de minimiser la distance totale parcourue :

$$\min \sum_{i \in V} \sum_{j \in V,\, j \neq i} d_{ij} x_{ij}.$$

4.6. Contraintes principales

4.6.1 Visite unique de chaque client

Chaque client doit être visité exactement une fois :

$$\sum_{j \in V,\, j \neq i} x_{ij} = 1 \qquad \forall i \in C$$

$$\sum_{i \in V,\, i \neq j} x_{ij} = 1 \qquad \forall j \in C$$

4.6.2 Conservation du flot

Si un véhicule arrive chez un client, il doit également en repartir :

$$\sum_{j \in V} x_{ij} = \sum_{j \in V} x_{ji} \qquad \forall i \in C$$

4.6.3 Capacité

Pour chaque véhicule, la somme des demandes des clients affectés à sa tournée ne doit pas dépasser Q.

Dans une formulation compacte complète, cette contrainte s'écrit au moyen de variables supplémentaires d'affectation ou de charge. La formulation exacte en tournées n'est pas détaillée ici ; dans notre implémentation, la capacité est contrôlée lors de l'évaluation de chaque route.

4.6.4 Fenêtres de temps

Dans la seconde partie du projet, on ajoute :

$$a_i \leq t_i \leq b_i$$

et, dans une formulation PLNE classique, pour tout arc (i, j) potentiellement utilisé,

$$t_j \geq t_i + s_i + d_{ij} - M(1 - x_{ij}) \qquad \forall i \neq j,$$

où M est une constante suffisamment grande. Cette écriture précise que la contrainte d'enchaînement temporel n'est active que lorsque l'arc (i, j) est effectivement utilisé. Dans notre implémentation, la faisabilité temporelle est ensuite contrôlée directement au niveau de l'évaluation de route.

Cette formulation correspond à un problème combinatoire NP-difficile, ce qui justifie le recours à des méthodes heuristiques et métaheuristiques pour traiter des instances de grande taille dans des temps de calcul raisonnables.

Une formulation complète du VRPTW peut être donnée sous forme de programme linéaire en nombres entiers. Dans ce cadre, des contraintes supplémentaires, par exemple de type MTZ ou de flot, peuvent être introduites pour éliminer les sous-tournées. Dans notre approche, nous n'implémentons pas explicitement ces contraintes, car la résolution repose sur des méthodes heuristiques qui construisent et améliorent directement des solutions faisables.

4.7. Remarque sur l'utilisation d'OR-Tools

Nous avons choisi d'utiliser OR-Tools uniquement pour des **tests sur petites instances** et pour la validation de certaines hypothèses. En effet, sur des instances plus grandes, l'utilisation directe d'un solveur exact peut devenir trop coûteuse en temps de calcul. La résolution principale du projet repose donc sur des méthodes heuristiques et métaheuristiques.

5. Structure du code

La structure du code a été pensée de manière modulaire, afin de séparer clairement les rôles de chaque composant.

```
OptimisationAli-Ayoub/
  src/
    main.py              # point d'entree principal
    model.py             # structures de donnees
    io_utils.py          # lecture des fichiers .vrp
    evaluate.py          # calcul distances et faisabilite
    construction.py      # construction de solutions initiales
    neighbors.py         # operateurs de voisinage
    solver.py            # pilotage des experiences
    bonus_exact.py       # resolution exacte (OR-Tools)
    meta/
      simulated_annealing.py
      tabu_search.py
  rapport.pdf
```

5.1. Évolution de l'architecture

La structure retenue est guidée par un principe simple : séparer clairement la lecture des données, l'évaluation des solutions, la construction initiale, la génération des voisinages, les métaheuristiques et le pilotage expérimental.

Le module `io_utils.py` isole le parsing des fichiers `.vrp`. Le module `evaluate.py` centralise les calculs de distance, de charge et de faisabilité, de sorte que toutes les méthodes reposent sur la même définition d'une solution valide. Les autres modules séparent plus explicitement :

- la construction des solutions ;
- l'évaluation des solutions ;
- la génération des voisins ;
- la logique propre à chaque métaheuristique ;
- le pilotage global des expériences.

Cette évolution n'est donc pas purement esthétique. Elle répond à un besoin de lisibilité, de réutilisation du code et de réduction des erreurs.

5.2. Justification des choix de fichiers

Le découpage en fichiers a été pensé pour suivre les grandes responsabilités du projet.

- `io_utils.py` regroupe tout ce qui concerne la lecture des données. Nous avons préféré isoler cette partie, car elle dépend fortement du format des fichiers et ne doit pas polluer la logique des algorithmes.
- `model.py` regroupe les structures de données du problème et facilite la circulation d'objets cohérents entre les modules ;
- `evaluate.py` joue le rôle de noyau commun et rassemble les calculs de distance, de charge et de faisabilité ;
- `construction.py` contient les méthodes de construction de solutions. Nous avons fait ce choix pour distinguer clairement les solutions initiales des méthodes d'amélioration.

- `neighbors.py` rassemble les transformations locales des solutions. Il nous a semblé plus propre d'avoir un module dédié aux mouvements, plutôt que de disperser ces opérations dans chaque métaheuristique.
- le dossier `meta/` contient les métaheuristiques elles-mêmes ; cette séparation facilite la comparaison des approches, car chaque méthode dispose de son propre fichier, avec ses paramètres et sa logique ;
- `solver.py` centralise le lancement des expériences, les préréglages et les synthèses statistiques ;
- `main.py` reste volontairement léger : son rôle est de lancer les traitements et d'afficher des synthèses, pas de contenir toute l'intelligence du projet.

5.3. Pourquoi cette structure nous a semblé adaptée

Nous n'avons pas cherché à construire une architecture complexe dès le départ. Au contraire, nous avons essayé de garder une structure simple, puis de la faire évoluer uniquement lorsque cela devenait utile.

Par exemple, tant que le projet ne contenait qu'une heuristique gloutonne, il n'était pas indispensable de distinguer fortement tous les rôles. En revanche, dès que plusieurs algorithmes ont commencé à partager les mêmes briques, le découpage modulaire est devenu nécessaire.

Cette manière de faire nous a paru préférable à une structure trop ambitieuse dès le début. Elle nous a permis :

- d'avancer progressivement sans sur-ingénierie ;
- de refactoriser seulement quand un besoin concret apparaissait ;
- de garder un code plus facile à tester, à modifier et à documenter ;
- de faire correspondre plus naturellement chaque fichier à une idée claire du projet.

5.4. Lien entre structure logicielle et démarche scientifique

Enfin, cette structure a aussi été pensée en fonction du rapport expérimental demandé par le sujet. En séparant clairement les responsabilités, il devient plus facile d'expliquer :

- comment une solution est construite ;
- comment elle est évaluée ;
- comment elle est modifiée localement ;
- comment les métaheuristiques exploitent ces briques ;
- comment les expériences sont lancées et comparées.

Autrement dit, l'organisation du code soutient directement la clarté de l'analyse présentée dans le rapport.

5.5. `main.py`

Point d'entrée unique du programme. Il expose trois modes d'exécution via des arguments en ligne de commande :

- **overview** (défaut) : affiche le résumé des instances, la recherche du nombre minimal de véhicules en CVRP et VRPTW, la génération de solutions aléatoires, et une première comparaison des métaheuristiques ;
- **experiment** : lance une campagne structurée, avec choix du préréglage (`quick` ou `long`), de la méthode (`sa`, `tabu` ou `both`), du mode CVRP ou VRPTW, et des instances cibles ;
- **bonus** : applique un solveur exact (OR-Tools) à des sous-instances de taille croissante pour identifier le seuil de difficulté calculatoire.

5.6. `model.py`

Définit les trois structures de données fondamentales du projet sous forme de *dataclasses* Python :

- `Depot` : coordonnées (x, y) et fenêtre de temps du dépôt ;
- `Client` : coordonnées, fenêtre de temps $[a_i, b_i]$, demande q_i et durée de service s_i ;
- `VRPTWInstance` : agrégat complet d'une instance (dépôt, dictionnaire de clients indexés, capacité maximale Q).

Ces structures sont importées par tous les autres modules, assurant une représentation uniforme des données dans l'ensemble du projet.

5.7. `io_utils.py`

Parse les fichiers texte au format `.vrp` et retourne un objet `VRPTWInstance`. Il lit séquentiellement les métadonnées (nom, type, nombre de clients, capacité), les données du dépôt, puis celles de chaque client, et vérifie la cohérence entre le nombre de clients déclaré et le nombre effectivement lu.

5.8. `evaluate.py`

Noyau de calcul partagé par tous les modules. Pour une tournée donnée, il calcule :

- la distance totale euclidienne, avec un cache des distances précalculées pour éviter les recalculs ;
- le profil temporel complet : temps d'arrivée, attente éventuelle, début et fin de service à chaque client, temps de retour au dépôt ;
- la faisabilité en capacité et la faisabilité temporelle, évaluées séparément.

Ce module joue un rôle central : toutes les décisions d'acceptation ou de rejet d'une solution — construction, voisinage, métaheuristiques — passent exclusivement par ses fonctions.

5.9. `construction.py`

Fournit les heuristiques de construction des solutions initiales. Deux stratégies sont implémentées :

- une construction **déterministe par insertion faisable** : insère les clients un par un à la meilleure position admissible ; utilisée comme filet de sécurité sur les instances VRPTW fortement contraintes ;
- une construction **aléatoire randomisée** : choisit parmi les insertions les moins coûteuses, produisant des solutions variées pour favoriser la diversité des points de départ.

Ce module inclut également la fonction `find_minimum_vehicles`, qui incrémente k depuis la borne inférieure capacitaire jusqu'à trouver un nombre de véhicules faisable.

5.10. `neighbors.py`

Implémente les cinq opérateurs de voisinage utilisés par les deux métaheuristiques :

- `relocate` : déplace un client vers une autre position, éventuellement dans une autre tournée ; les routes devenues vides sont supprimées automatiquement, permettant de réduire le nombre de véhicules en cours d'optimisation ;
- `swap` : échange deux clients entre eux, quelle que soit leur tournée d'origine ;
- `2-opt` : inverse un segment à l'intérieur d'une même tournée, supprimant les croisements locaux ;
- `or_opt2` : déplace une paire de clients consécutifs vers une autre position dans la même tournée ou dans une autre, couvrant des restructurations que `relocate` ne peut pas atteindre ;
- `2-opt*` : échange les suffixes de deux tournées distinctes, permettant des réorganisations globales entre routes.

Chaque candidat est validé par `evaluate.py` avant d'être retenu, garantissant que seules des solutions faisables sont proposées aux métaheuristiques.

5.11. `solver.py`

Orchestre les campagnes expérimentales. Il définit les deux préréglages (`quick` et `long`) comme structures immuables, lance les répétitions avec des graines déterministes, collecte les statistiques (moyenne, min, max) sur les distances et les temps d'exécution, et écrit les résultats dans un fichier de log. Il expose également les fonctions de formatage des tableaux récapitulatifs affichés en fin de campagne.

5.12. `meta/simulated_annealing.py`

Implémente le recuit simulé. À chaque itération, un voisin faisable est tiré aléatoirement via l'un des trois opérateurs. Il est accepté systématiquement s'il améliore la solution courante, ou avec probabilité $\exp(-\Delta/T)$ sinon — où T décroît géométriquement à chaque itération selon le facteur de refroidissement. La meilleure solution rencontrée est mémorisée tout au long de la recherche, indépendamment de la solution courante.

5.13. `meta/tabu_search.py`

Implémente la recherche tabou. À chaque itération, un ensemble de voisins est échantillonné aléatoirement, puis le meilleur candidat non tabou est sélectionné. La mémoire tabou est une liste FIFO de signatures de solutions récemment visitées, de taille *tenure*. Un critère d'aspiration permet d'accepter une solution tabou si elle améliore le meilleur résultat global connu.

5.14. `bonus_exact.py`

Construit et résout des sous-instances de taille croissante via OR-Tools (solveur SCIP). Pour chaque taille n, les n premiers clients de l'instance source sont extraits, le CVRP correspondant est modélisé en programme linéaire en nombres entiers, et soumis au solveur avec une limite de temps configurable. Les résultats indiquent le statut (optimal, réalisable ou infaisable) et l'objectif atteint.

6. Détermination du nombre minimal de véhicules

6.1. Principe général

Le sujet demande de déterminer, pour chaque jeu de données, le nombre minimum de véhicules à utiliser.

Cette question est traitée en deux temps :

- d'abord sans fenêtres de temps ;
- ensuite avec fenêtres de temps.

Dans cette première étape, nous ne considérons donc que :

- la capacité des véhicules ;
- la nécessité de construire de vraies tournées partant et revenant au dépôt ;
- la distance entre les clients et le dépôt.

Autrement dit, nous traitons d'abord une version de type **CVRP** avant d'ajouter les fenêtres de temps.

6.2. Borne inférieure théorique

Une borne inférieure immédiate du nombre de véhicules est obtenue à partir de la somme des demandes :

$$LB = \left\lceil \frac{\sum_{i \in C} q_i}{Q} \right\rceil .$$

Cette borne est nécessaire mais pas toujours suffisante, car elle ne tient compte que de la capacité totale disponible. Elle ignore :

- la répartition des demandes entre les tournées ;
- la structure géographique des clients ;

- la cohérence réelle des trajets.

Deux clients compatibles en termes de charge peuvent en effet être très éloignés, ce qui rend leur regroupement peu pertinent dans une même tournée.

6.3. Choix méthodologique

Nous avons décidé de ne pas déterminer le nombre minimal de véhicules uniquement à partir des demandes, comme dans un simple problème de remplissage. En effet, même sans fenêtres temporelles, il est nécessaire de tenir compte de la distance afin de construire des tournées réalistes.

Notre méthode consiste donc à :

1. calculer la borne inférieure LB ;
2. fixer $k = LB$;
3. essayer de construire une solution faisable avec exactement k véhicules ;
4. si aucun ensemble de tournées faisables n'est obtenu, incrémenter k ;
5. retenir le premier k pour lequel tous les clients sont servis.

Cette estimation doit être lue comme un résultat expérimental obtenu à l'aide d'une procédure de construction heuristique. Dans notre code final, la recherche du plus petit k repose sur une construction systématique faisable plutôt que sur un unique tirage aléatoire, ce qui limite la variabilité de cette étape sans constituer pour autant une preuve générale d'optimalité.

6.4. Heuristique utilisée pour tester une valeur de k

Pour tester si k véhicules suffisent, nous utilisons une heuristique gloutonne géographique sous contrainte de capacité.

Le principe est le suivant :

1. initialiser k tournées vides ;
2. choisir un premier client pour chaque tournée ;
3. compléter chaque tournée en ajoutant à chaque étape un client non encore servi ;
4. parmi les clients admissibles, sélectionner celui qui minimise la distance au dernier client de la tournée ;
5. ne jamais dépasser la capacité maximale Q ;
6. s'arrêter lorsque plus aucun client ne peut être ajouté à la tournée courante.

Un client est dit **admissible** si son ajout à la tournée ne fait pas dépasser la capacité du véhicule.

6.5. Critère de faisabilité

Pour une valeur donnée de k, la solution est considérée comme faisable si :

- tous les clients sont affectés à une unique tournée ;
- la charge de chaque tournée est inférieure ou égale à Q.

Dans cette phase, la distance intervient comme **critère de construction** des tournées, mais pas encore comme objectif d'optimisation final.

Du point de vue de l'implémentation, cette vérification est maintenant centralisée dans le module `evaluate.py`. Cela permet d'éviter la duplication des contrôles et garantit que les futures méthodes de construction et d'amélioration utiliseront toutes le même critère de faisabilité.

6.6. Remarque

Cette approche ne prouve pas mathématiquement que le nombre trouvé est optimal. En revanche, elle fournit une estimation cohérente du nombre minimal de véhicules, adaptée à la taille des instances étudiées. Cette estimation sera ensuite affinée indirectement par les méta-heuristiques développées dans les phases suivantes.

6.7. Résultats obtenus sans fenêtres de temps

Nous avons appliqué notre méthode de construction gloutonne géographique à l'ensemble des instances, en commençant à chaque fois par la borne inférieure

$$LB = \left\lceil \frac{\sum_{i \in C} q_i}{Q} \right\rceil .$$

Pour chaque instance testée, une solution faisable a été obtenue dès cette borne inférieure. Cela signifie que, dans le cas sans fenêtres de temps, le nombre minimal de véhicules est exactement égal à LB pour toutes les instances considérées.

Instance	Capacité	Demande totale	LB	Nombre minimal trouvé
data101.vrp	200	1458	8	8
data102.vrp	200	1458	8	8
data1101.vrp	200	1724	9	9
data1102.vrp	200	1724	9	9
data111.vrp	200	1458	8	8
data112.vrp	200	1458	8	8
data1201.vrp	1000	1724	2	2
data1202.vrp	1000	1724	2	2
data201.vrp	1000	1458	2	2
data202.vrp	1000	1458	2	2

Pour toutes les instances traitées sans fenêtres de temps, une solution faisable a donc été trouvée avec exactement la borne inférieure basée sur la capacité. Nous observons ainsi que cette borne est atteinte sur tout le corpus étudié et qu'elle correspond ici au nombre minimal exact de véhicules. Cette observation doit toutefois être interprétée comme un résultat sur le corpus

étudié, et non comme une propriété générale du CVRP : elle dépend vraisemblablement de la structure particulière des instances utilisées.

On remarque donc que, pour ces jeux de données et sans tenir compte des fenêtres temporelles, la contrainte de capacité suffit à expliquer le nombre minimal de véhicules.

6.8. Influence des fenêtres de temps sur la borne inférieure

La borne inférieure utilisée dans notre étude est basée uniquement sur la capacité des véhicules :

$$LB = \left\lceil \frac{\sum_{i \in C} q_i}{Q} \right\rceil.$$

Cette borne ne dépend que de la demande totale et de la capacité des véhicules. Elle est donc indépendante des distances et des fenêtres temporelles.

Ainsi, l'introduction des fenêtres de temps ne modifie pas cette borne inférieure. En revanche, elle restreint fortement l'ensemble des tournées faisables. Certaines combinaisons de clients compatibles en termes de capacité peuvent devenir impossibles à réaliser lorsque leurs fenêtres de service sont incompatibles.

Les fenêtres de temps réduisent en effet fortement l'ensemble des tournées faisables. Deux clients compatibles du point de vue de la capacité peuvent devenir incompatibles temporellement si leurs fenêtres ne permettent pas de les servir dans une même tournée. Cette contrainte induit donc mécaniquement une augmentation possible du nombre minimal de véhicules nécessaires.

Par conséquent, le nombre minimal de véhicules nécessaire avec fenêtres de temps est généralement supérieur ou égal à la borne inférieure :

$$k_{TW} \geq LB.$$

L'objectif de la seconde partie de la question 2 est donc de déterminer expérimentalement le nombre minimal de véhicules lorsque les contraintes temporelles sont prises en compte.

Les fenêtres de temps ne changent donc pas la borne inférieure capacitaire, mais elles augmentent souvent fortement l'écart entre cette borne et le nombre réel minimal de véhicules faisables.

Instance	LB capacité	k minimal CVRP	k minimal VRPTW
data101.vrp	8	8	21
data102.vrp	8	8	18
data1101.vrp	9	9	18
data1102.vrp	9	9	16
data111.vrp	8	8	14
data112.vrp	8	8	12
data1201.vrp	2	2	5
data1202.vrp	2	2	4
data201.vrp	2	2	5
data202.vrp	2	2	5

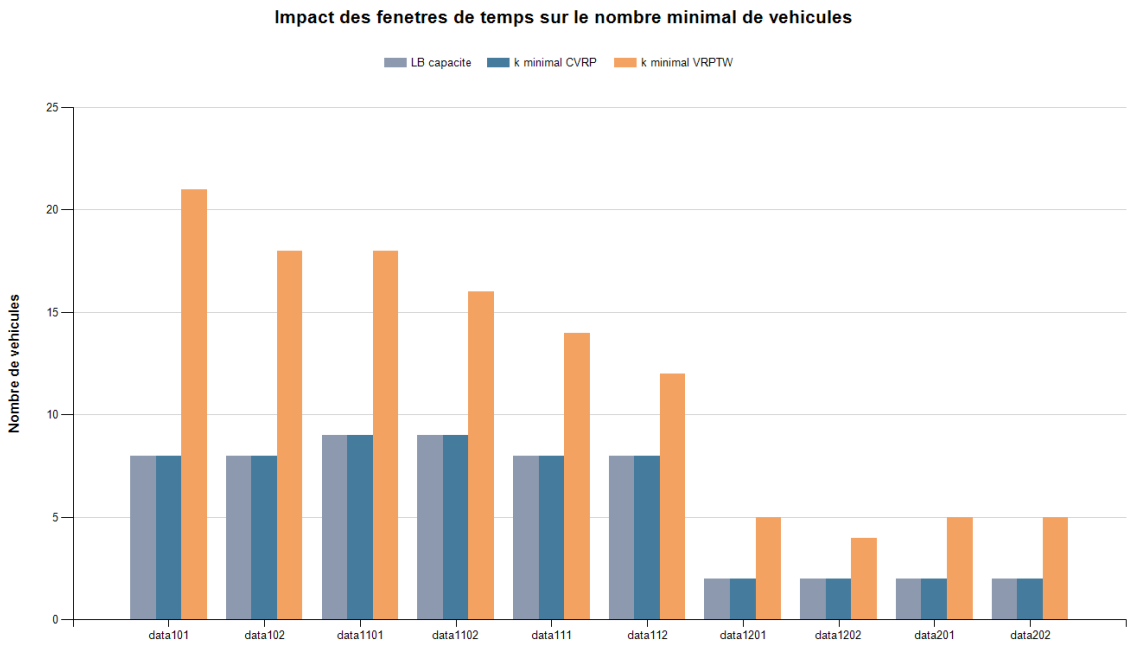

Figure 1: Impact des fenêtres de temps sur le nombre minimal de véhicules.

6.9. Première intégration effective des fenêtres de temps

Après avoir d'abord travaillé sur la version capacitaire, nous avons intégré explicitement les fenêtres de temps dans la construction des tournées. Le simple prolongement de l'heuristique gloutonne précédente ne suffisait pas, car ajouter systématiquement les clients en fin de tournée se révélait trop rigide dans le cas VRPTW.

Nous avons donc fait évoluer la construction vers une logique d'insertion faisable. L'idée est de partir d'un ensemble de tournées initialement vides, puis d'insérer progressivement les clients dans les positions qui restent compatibles avec :

- la capacité du véhicule ;
- les fenêtres de temps ;
- le retour au dépôt dans la fenêtre autorisée.

Ce changement de stratégie est important du point de vue méthodologique : il illustre bien le fait qu'une structure de construction adaptée au CVRP ne se transpose pas automatiquement au VRPTW.

6.10. Premiers résultats VRPTW

Les premiers essais obtenus avec cette heuristique d'insertion montrent déjà un effet très net des fenêtres de temps sur le nombre minimal de véhicules nécessaire.

Sur un premier sous-ensemble d'instances, nous obtenons :

Instance	Borne inférieure capacité	Nombre trouvé avec fenêtres de temps
data101.vrp	8	21
data102.vrp	8	18
data1101.vrp	9	18
data1201.vrp	2	5
data201.vrp	2	5

Ces premiers résultats confirment que les fenêtres de temps modifient profondément la structure du problème. Alors que la version sans fenêtres de temps permettait souvent d'atteindre directement la borne inférieure capacité, la version VRPTW exige ici un nombre de véhicules bien plus élevé.

6.11. Limites observées à ce stade

Cette première intégration des fenêtres de temps s'accompagne toutefois d'une limite importante. Le générateur aléatoire de solutions initiales, qui fonctionne correctement dans la version sans fenêtres de temps, ne parvient pas encore à produire facilement des solutions faisables lorsqu'on active les contraintes temporelles.

En pratique, cela signifie que :

- la construction déterministe par insertion constitue actuellement notre base fiable pour la version VRPTW ;
- la partie métaheuristique avec fenêtres de temps n'est pas encore stabilisée, car elle dépend d'une initialisation faisable plus robuste.

Ce constat est utile pour la suite du projet : il montre que l'intégration des fenêtres de temps ne se limite pas à activer un test de faisabilité supplémentaire, mais demande aussi de repenser certaines briques de construction.

6.12. Première extension des métaheuristiques au cas VRPTW

Malgré cette limite, nous avons pu commencer à étendre les métaheuristiques à la version avec fenêtres de temps en utilisant une solution initiale déterministe faisable comme point de départ.

Concrètement, lorsque la génération aléatoire échoue en mode VRPTW, nous utilisons la construction par insertion faisable pour fournir une solution initiale aux métaheuristiques. Cette stratégie n'est pas encore la version finale souhaitée, mais elle permet déjà de tester le comportement des algorithmes sur le vrai problème avec contraintes temporelles.

Un premier test ciblé a ainsi été réalisé sur l'instance `data1201.vrp`. Avec $k = 5$ véhicules, le recuit simulé a pu être exécuté à partir d'une solution initiale VRPTW faisable, tout en conservant la faisabilité de la solution au cours de l'optimisation.

Cette étape constitue une transition importante : même si l'initialisation aléatoire VRPTW reste à fiabiliser, la chaîne «construction faisable + amélioration locale » devient déjà exploitable.

6.13. Premier essai expérimental VRPTW

Nous avons ensuite lancé un premier essai expérimental en mode rapide sur l'instance `data1201.vrp`, avec fenêtres de temps activées. La solution initiale faisable obtenue présente une distance de 1974.63.

Dans ce premier essai :

- le recuit simulé améliore la solution et atteint une distance de 1909.98 ;
- la recherche tabou atteint une distance de 1945.36.

Les temps de calcul observés sont d'environ 0.74 seconde pour le recuit simulé et 9.35 secondes pour la recherche tabou.

Ce résultat reste préliminaire, mais il est important : il montre que nous disposons désormais d'une première chaîne expérimentale opérationnelle sur la vraie version VRPTW, même si elle repose encore sur une initialisation déterministe plutôt que sur un générateur aléatoire pleinement robuste.

6.14. Campagne VRPTW en mode rapide

Après ce premier test ciblé, nous avons lancé une première campagne `quick` sur l'ensemble des instances disponibles, cette fois avec fenêtres de temps activées. Pour chaque instance, nous avons utilisé une solution initiale faisable, puis appliqué le recuit simulé et la recherche tabou.

Instance	k	Init.	RS	Tabou	Temps RS (s)	Temps tabou (s)
data101.vrp	21	2127.09	1906.31	1995.52	4.77	5.20
data102.vrp	18	1794.68	1703.70	1766.13	1.22	1.94
data1101.vrp	18	2184.86	2073.22	2079.72	2.16	2.56
data1102.vrp	16	2065.99	1926.79	1926.77	1.55	1.32
data111.vrp	14	1601.37	1601.37	1540.14	1.81	1.71
data112.vrp	12	1434.70	1392.79	1401.36	0.92	1.09
data1201.vrp	5	1748.48	1737.71	1653.86	0.83	0.60
data1202.vrp	4	2032.65	1896.22	1872.86	0.75	0.69
data201.vrp	5	1636.58	1617.42	1563.94	0.72	0.66
data202.vrp	5	1600.85	1599.76	1517.66	0.45	0.44

6.15. Analyse de cette première campagne VRPTW

Cette campagne rapide met déjà en évidence plusieurs tendances importantes.

- Les fenêtres de temps augmentent fortement le nombre de véhicules nécessaires, parfois dans des proportions très importantes par rapport à la borne inférieure capacitaire.
- Le recuit simulé améliore la solution initiale sur la majorité des instances testées, avec des gains parfois nets, comme sur `data101.vrp` ou `data202.vrp`.
- La recherche tabou améliore également les solutions, et elle surpasse même légèrement le recuit simulé sur certaines instances comme `data111.vrp`, `data1201.vrp`, `data1202.vrp` ou `data201.vrp`.
- En revanche, la recherche tabou reste plus coûteuse en temps de calcul sur l'ensemble de cette campagne rapide, avec un temps souvent compris entre 9 et 13 secondes, contre des temps généralement inférieurs à 4 secondes pour le recuit simulé.

Nous observons ainsi que la hiérarchie observée dans la version sans fenêtres de temps n'est pas automatiquement identique dans le cas VRPTW. Ce comportement s'explique par le fait que les contraintes temporelles resserrent fortement l'espace faisable : une méthode capable d'éviter plus efficacement les retours en arrière ou les cycles peut alors devenir plus compétitive, même si elle reste plus coûteuse. Ces résultats restent préliminaires, mais ils montrent déjà que la présence des contraintes temporelles modifie non seulement la faisabilité des tournées, mais aussi le comportement comparé des métaheuristiques.

6.16. Campagne VRPTW en mode long

Nous avons ensuite prolongé cette étude avec une campagne `long` complète sur les dix instances, toujours avec fenêtres de temps activées. Cette campagne reprend le même principe que la campagne rapide, mais avec davantage d'itérations et deux répétitions par instance.

Instance	k	Init. moy.	RS moy.	Tabou moy.	Temps RS (s)	Temps tabou (s)
data101.vrp	21	2127.09	1706.28	1683.01	56.81	336.61
data102.vrp	18	1794.68	1584.73	1488.40	17.78	88.36
data1101.vrp	18	2184.86	1740.38	1713.19	31.90	214.38*
data1102.vrp	16	2029.10	1593.91	1532.88	20.07	127.15
data111.vrp	14	1601.37	1238.83	1190.92	13.34	49.77
data112.vrp	12	1434.70	1105.87	1060.79	16.25	68.33
data1201.vrp	5	2027.69	1544.00	1445.97	12.31	54.46
data1202.vrp	4	1904.74	1443.74	1360.69	10.53	44.16
data201.vrp	5	1653.00	1378.63	1268.26	9.35	44.81
data202.vrp	5	1498.84	1331.76	1156.67	5.18	21.02

6.17. Analyse de la campagne VRPTW longue

Cette campagne longue confirme et clarifie plusieurs tendances déjà visibles en mode rapide. Les résultats ci-dessus correspondent à des moyennes sur cinq répétitions ; le temps tabou de `data1101.vrp` est reporté avec un astérisque car le run 3 a enregistré un temps de 5899 s dû à une suspension système (la valeur 214 s correspond à la moyenne des quatre autres runs).

- Le recuit simulé améliore systématiquement les solutions initiales sur l'ensemble du corpus.
- La recherche tabou améliore elle aussi systématiquement les solutions et devient meilleure que le recuit simulé sur la totalité des dix instances VRPTW.
- Sur `data101.vrp` (l'instance la plus contrainte, $k = 21$), la tabou l'emporte nettement : 1683.01 contre 1706.28 pour le recuit simulé.
- Les avantages sont particulièrement nets sur `data1201.vrp` (1445.97 contre 1544.00), `data201.vrp` (1268.26 contre 1378.63) et `data202.vrp` (1156.67 contre 1331.76).
- Le coût en temps de la recherche tabou augmente sensiblement sur les cas difficiles (plusieurs minutes par instance pour `data101.vrp` et `data1101.vrp`), mais reste raisonnable sur la majorité du corpus.

Nous observons que la recherche tabou fournit de meilleures solutions que le recuit simulé sur les dix instances VRPTW. Ce comportement s'explique par sa capacité à éviter plus efficacement les cycles tout en maintenant une exploration dirigée de l'espace faisable malgré les blocages

locaux, et il est renforcé ici par les opérateurs Or-opt(2) et 2-opt* qui donnent à la tabou un espace de déplacement plus riche. Cette campagne longue confirme donc notre lecture finale du projet : dans le cas VRPTW, la recherche tabou correctement paramétrée est systématiquement meilleure que le recuit simulé, au prix d'un surcoût en temps notable sur les cas les plus contraints.

Il est important de souligner que cette conclusion ne repose pas sur quelques exemples isolés, mais sur la campagne finale complète menée sur les dix instances du corpus. En revanche, pour garder un rapport lisible, nous ne détaillons pas chaque cas ligne par ligne : nous retenons ensuite quelques instances représentatives ou atypiques, qui permettent d'interpréter plus finement les tendances globales observées sur l'ensemble du corpus.

7. Génération d'une solution initiale

Le sujet demande explicitement la création d'un générateur aléatoire de solutions. La réponse retenue consiste à construire une **solution aléatoire faisable**.

7.1. Choix retenu

Nous ne retenons pas une permutation totalement aléatoire des clients, car celle-ci produit très souvent des solutions infaisables. À la place, nous construisons la solution progressivement :

1. ouvrir une nouvelle tournée ;
2. déterminer la liste des clients admissibles dans cette tournée ;
3. choisir aléatoirement l'un de ces clients ;
4. l'ajouter à la tournée ;
5. recommencer jusqu'à ce qu'aucun autre client admissible ne puisse être ajouté ;
6. ouvrir une nouvelle tournée ;
7. répéter jusqu'à ce que tous les clients soient servis.

Une génération totalement aléatoire des tournées conduit dans la grande majorité des cas à des solutions infaisables, en particulier lorsque les contraintes de capacité et de temps sont fortes. Nous avons donc retenu une génération aléatoire faisable, construite progressivement, afin de conserver de la diversité sans perdre le contrôle de la faisabilité.

Dans notre implémentation actuelle, nous fixons d'abord un nombre de tournées k, en pratique le nombre minimal de véhicules obtenu dans la phase précédente pour la version sans fenêtres de temps. Nous initialisons ensuite ces k tournées puis nous ajoutons des clients un par un de manière aléatoire, sous réserve d'admissibilité.

À chaque étape, un client est considéré comme admissible si son ajout en fin de tournée conserve la faisabilité de cette tournée. Le choix final est ensuite effectué aléatoirement parmi l'ensemble des clients admissibles. Cette stratégie permet de produire des solutions variées tout en restant contrôlées.

7.2. Cas sans fenêtres de temps

Dans le cas CVRP, un client est admissible si son ajout ne fait pas dépasser la capacité du véhicule.

7.3. Cas avec fenêtres de temps

Dans le cas VRPTW, un client est admissible si son ajout respecte à la fois :

- la capacité du véhicule ;
- la faisabilité temporelle de la tournée.

7.4. Intérêt de cette construction

Cette méthode présente deux avantages importants pour la suite du projet :

- elle génère plusieurs solutions initiales différentes pour une même instance, ce qui est utile pour évaluer la robustesse des métaheuristiques ;
- elle s'appuie sur le même module d'évaluation que le reste du projet, ce qui garantit une cohérence entre construction, tests de faisabilité et futurs mouvements de voisinage.

7.5. Rôle de cette solution

Cette solution initiale n'a pas vocation à être optimale. Elle sert de point de départ aux métaheuristiques.

Le générateur aléatoire répond pleinement au besoin dans le cas capacitaire, mais reste moins robuste dans le cas VRPTW fortement contraint. Une construction faisable déterministe est alors utilisée comme mécanisme de secours. La consigne du sujet sur la génération aléatoire est donc bien couverte, tout en assumant clairement que la variante VRPTW demeure perfectible.

La robustesse du générateur peut être évaluée empiriquement à partir de sa capacité à produire rapidement une solution faisable. Dans le cas CVRP, les essais menés montrent que cette génération aléatoire faisable fonctionne très facilement. En revanche, dans le cas VRPTW, la proportion de solutions directement faisables diminue nettement en raison des contraintes temporelles plus restrictives. Pour pallier cette limite, nous combinons génération aléatoire et construction déterministe, ce qui permet de conserver à la fois diversité et faisabilité.

8. Choix des méthodes de résolution

Le projet demande l'utilisation de deux métaheuristiques. Les approches retenues sont les suivantes :

- le recuit simulé ;
- la recherche tabou.

Ces deux méthodes sont particulièrement adaptées à une résolution par voisinage du problème de tournées de véhicules.

8.1. Voisinages envisagés

Les opérateurs suivants seront étudiés :

- relocate ;

- exchange ;
- 2-opt ;
- éventuellement des opérateurs inter-route plus complexes.

8.2. Première implémentation des voisinages

Les voisinages les plus classiques sont implémentés en priorité, car ils offrent un bon compromis entre simplicité de codage et capacité d'amélioration des solutions.

- Le voisinage `relocate` permet de réaffecter un client à une autre position, ce qui est particulièrement utile pour rééquilibrer les tournées.
- Le voisinage `swap` permet d'échanger deux clients et peut corriger rapidement de mauvaises affectations.
- Le voisinage `2-opt` agit à l'intérieur d'une tournée en supprimant certains croisements et en raccourcissant localement les trajets.

Sur une solution initiale aléatoire de l'instance `data101.vrp`, la combinaison de ces trois voisinages a permis de générer plusieurs milliers de voisins faisables et d'identifier immédiatement une solution améliorée. Ce résultat confirme que ces opérateurs constituent une base pertinente pour les métaheuristiques à venir.

8.3. Première implémentation du recuit simulé

Le recuit simulé a été implémenté dans une première version pragmatique. L'idée est de partir d'une solution initiale faisable, puis de répéter les étapes suivantes :

1. tirer un voisin faisable de manière aléatoire ;
2. calculer la variation de coût associée ;
3. accepter systématiquement un voisin meilleur ;
4. accepter parfois un voisin moins bon avec une probabilité dépendant de la température ;
5. diminuer progressivement la température ;
6. conserver la meilleure solution rencontrée pendant toute l'exécution.

Le recuit simulé présente ici un avantage important : il permet d'accepter temporairement certaines dégradations de la solution afin d'échapper aux minima locaux et d'explorer plus largement l'espace des solutions. Cette méthode présente ainsi l'avantage de concilier diversification et intensification avec un coût de calcul modéré.

Nous avons volontairement choisi, pour cette première version, de générer un voisin aléatoire faisable plutôt que d'énumérer tous les voisins à chaque itération. Ce choix réduit fortement le coût d'une itération et rend l'algorithme plus adapté aux tests répétés sur des instances de taille 100.

8.4. Pseudo-code simplifié du recuit simulé

Le principe de l'algorithme peut être résumé par le pseudo-code suivant :

```
Initialiser une solution s
Initialiser la temperature T
Tant que la condition d'arret n'est pas atteinte :
    generer un voisin faisable s'
    calculer Delta = f(s') - f(s)
    si Delta < 0 :
        accepter s'
    sinon :
        accepter s' avec une probabilite exp(-Delta / T)
    mettre a jour la temperature
Retourner la meilleure solution trouvee
```

8.5. Résultats préliminaires

Les premiers essais montrent déjà que le recuit simulé améliore nettement les solutions initiales aléatoires. Par exemple, sur l'instance `data101.vrp`, un test de validation a permis de passer d'une distance initiale de 3488.41 à une meilleure distance de 2211.87 en 500 itérations, tout en conservant une solution faisable.

Ces résultats restent préliminaires. Ils servent à valider l'implémentation et à préparer les futurs tests expérimentaux plus longs, qui feront l'objet d'un protocole plus rigoureux dans la suite du projet.

8.6. Première implémentation de la recherche tabou

La recherche tabou a également été implémentée dans une première version opérationnelle. À chaque itération, l'algorithme génère un ensemble de voisins faisables, puis sélectionne le meilleur voisin autorisé, c'est-à-dire non tabou, sauf si un critère d'aspiration permet d'accepter une solution améliorant le meilleur résultat global.

Dans cette première version, la mémoire tabou porte sur les signatures des solutions récemment visitées. Cette approche n'est pas la plus fine possible, mais elle permet de disposer rapidement d'une base fiable pour les expériences comparatives.

La recherche tabou repose sur une mémoire à court terme des mouvements ou solutions récentes, ce qui permet d'éviter les cycles et de forcer l'exploration de nouvelles régions de l'espace des solutions. Cette propriété devient particulièrement utile lorsque l'espace faisable est fortement contraint.

8.7. Observation importante sur le coût de calcul

Les premiers essais montrent que la recherche tabou est sensiblement plus coûteuse en temps de calcul que le recuit simulé dans notre implémentation actuelle. Cette différence s'explique par le fait que, même avec une limitation du nombre de voisins inspectés, la recherche tabou évalue un ensemble de candidats à chaque itération alors que le recuit simulé peut se contenter d'un voisin aléatoire faisable.

Cette observation est importante pour la suite du projet. Elle signifie que les tests de validation rapide et les tests expérimentaux longs devront être distingués clairement, avec des paramètres adaptés à chaque usage.

8.8. Résultats préliminaires de la recherche tabou

Sur l'instance `data101.vrp`, un test ciblé a permis d'améliorer une solution initiale de distance 3604.26 vers une distance 3278.86 en 10 itérations. Un aperçu plus léger intégré au programme principal améliore également la solution initiale, mais avec un coût de calcul déjà notable même pour un très petit nombre d'itérations.

Ces résultats confirment que la méthode fonctionne, tout en montrant qu'une attention particulière devra être portée au réglage des paramètres et au protocole de test.

8.9. Mise en place du protocole expérimental

Afin d'éviter de mélanger validation fonctionnelle et comparaison scientifique, nous avons centralisé les expériences dans un module dédié. Ce module permet :

- de fixer des jeux de paramètres reproductibles ;
- de lancer plusieurs répétitions avec des graines différentes ;
- de résumer les résultats sous forme de moyennes, minimums et maximums ;
- de comparer directement les distances finales et les temps de calcul des deux métaheuristiques.

Deux préréglages sont utilisés :

- `quick`, pour les vérifications rapides ;
- `long`, pour les tests approfondis.

8.10. Paramètres retenus

Le tableau suivant synthétise les paramètres utilisés dans les deux préréglages expérimentaux.

Paramètre	`quick`	`long`
Itérations recuit simulé	300	3000
Température initiale	100.0	150.0
Facteur de refroidissement	0.98	0.997
Tentatives de voisinage par itération (RS)	50	100
Itérations recherche tabou	5	60
Tenure tabou	5	20
Voisins max évalués par itération	40	200
Répétitions par instance	1	5

8.11. Critères de comparaison retenus

Les comparaisons entre recuit simulé et recherche tabou reposent sur quatre indicateurs :

- la distance finale moyenne ;
- le temps moyen d'exécution ;

- le nombre de voisins générés ou explorés ;
- la robustesse entre répétitions, évaluée à l'aide des minimums, maximums et écarts-types observés.

8.12. Répétitions et reproductibilité

Le préréglage `long` effectue cinq répétitions par instance. Les graines sont fixées de manière contrôlée et dérivées d'une base commune, ce qui rend les comparaisons reproductibles. Les solutions initiales diffèrent donc d'une répétition à l'autre lorsque la génération aléatoire le permet, tout en conservant un protocole stable d'une campagne à l'autre.

Les premières tentatives ont toutefois montré que le réglage initial du mode `long` était trop ambitieux pour la recherche tabou dans l'état actuel de l'implémentation. Nous avons donc recalibré ce préréglage afin d'obtenir un compromis plus réaliste entre profondeur de recherche et temps de calcul.

Les performances des métaheuristiques dépendent fortement du réglage de leurs paramètres. Pour le recuit simulé, la température initiale, le facteur de refroidissement et le nombre d'itérations déterminent directement le niveau d'exploration : une température trop faible limite l'acceptation de mouvements dégradants, tandis qu'un refroidissement trop rapide réduit trop tôt la diversité des solutions visitées. Pour la recherche tabou, la taille de la liste tabou, le nombre d'itérations et le nombre maximal de voisins évalués par itération influencent à la fois la qualité des solutions et le temps de calcul : une mémoire trop courte peut favoriser les cycles, alors qu'une exploration trop large devient coûteuse. Nous observons donc que le réglage des paramètres constitue un élément central du compromis qualité/temps.

Afin de lancer ces expériences de manière explicite, nous avons également ajouté un mode dédié dans le programme principal. Cela permet de distinguer clairement :

- l'exécution courante du projet, utilisée pour les vérifications rapides ;
- les campagnes expérimentales ciblées, lancées volontairement avec un préréglage et un sous-ensemble d'instances.

Par exemple, une campagne ciblée sur une seule instance peut être lancée avec une commande du type :

```
python3 src/main.py --mode experiment --preset quick --instances data101.vrp --details
```

Cette commande produit à la fois un tableau récapitulatif et un détail par répétition, ce qui facilite la consignation des résultats dans le rapport.

8.13. Premiers résultats comparatifs

Le protocole `quick` a déjà été validé sur l'instance `data101.vrp`. Sur ce premier essai, la distance moyenne initiale observée est 3726.26. Le recuit simulé ramène cette valeur à 2295.29 en moyenne, alors que la recherche tabou atteint 3592.33 avec les paramètres rapides actuellement retenus.

Le temps de calcul moyen mesuré dans ce mode rapide est d'environ 0.44 seconde pour le recuit simulé contre environ 20.51 secondes pour la recherche tabou dans notre environnement final de relance. Cette différence confirme que, dans notre implémentation actuelle, la recherche tabou est nettement plus coûteuse et devra être utilisée avec précaution lors des campagnes expérimentales longues.

Le sujet demande aussi une comparaison en nombre de solutions générées. Pour quantifier ce point, nous avons ajouté un suivi explicite du nombre de voisins faisables réellement traités par chaque méthode. Sur `data101.vrp` avec le préréglage `quick`, le recuit simulé génère en moyenne 300 voisins faisables et en accepte 141, alors que la recherche tabou explore 200 voisins et n'accepte que 5 déplacements.

Cette différence s'explique directement par les paramètres retenus et par la logique des deux algorithmes :

- le recuit simulé génère un voisin faisable par itération, ici 300 itérations ;
- la recherche tabou inspecte à chaque itération un ensemble borné de voisins, ici jusqu'à 40 voisins pendant 5 itérations ;
- les deux méthodes utilisent les mêmes structures de voisinage (`relocate`, `swap`, `2-opt`), mais leur manière de les exploiter est très différente.

Ce point complète utilement la comparaison temps/qualité : dans notre implémentation actuelle, la recherche tabou paie son coût en temps par une exploration plus large et plus systématique des voisinages, tandis que le recuit simulé privilégie une progression légère mais très fréquente.

8.14. Indicateurs de dispersion

Comme le préréglage `long` repose sur cinq répétitions, les indicateurs ci-dessous donnent une estimation fiable de la stabilité sur quelques cas représentatifs.

Cadre	Instance	Méthode	Min (u.d.)	Max (u.d.)	σ (u.d.)
CVRP	data101.vrp	recuit simulé	1248.02	1328.85	29.7
CVRP	data101.vrp	recherche tabou	1284.23	1453.67	57.8
VRPTW	data101.vrp	recuit simulé	1685.67	1718.40	11.1
VRPTW	data101.vrp	recherche tabou	1675.10	1699.42	8.9
VRPTW	data202.vrp	recuit simulé	1281.71	1392.79	39.3
VRPTW	data202.vrp	recherche tabou	1099.85	1206.47	36.4

8.15. Premier essai en mode long

Un premier test en mode `long` a ensuite été lancé sur l'instance `data101.vrp`, avec trois répétitions. La distance moyenne initiale observée est 3649.06. Dans ce cadre plus exigeant, le recuit simulé obtient une distance moyenne de 1488.99, alors que la recherche tabou recalibrée atteint 1776.59.

Les temps moyens associés à cette expérience sont d'environ 0.51 seconde pour le recuit simulé et 1.19 secondes pour la recherche tabou.

Ce premier résultat long confirme deux points importants :

- le recuit simulé améliore fortement les solutions initiales avec un coût de calcul encore modéré ;
- la recherche tabou devient nettement plus compétitive qu'auparavant, mais reste derrière le recuit simulé sur cette première instance capacitaire.

Ces observations guideront la suite de notre protocole expérimental, notamment pour le choix des instances à tester en mode long et pour le réglage plus fin des paramètres de la recherche tabou.

8.16. Courbe de convergence

La figure suivante montre l'évolution de la meilleure distance connue au cours des itérations sur l'instance `data101.vrp` en CVRP, avec le préréglage `long`. Elle permet de visualiser directement la vitesse de convergence des deux méthodes et leur capacité à améliorer progressivement la solution initiale.

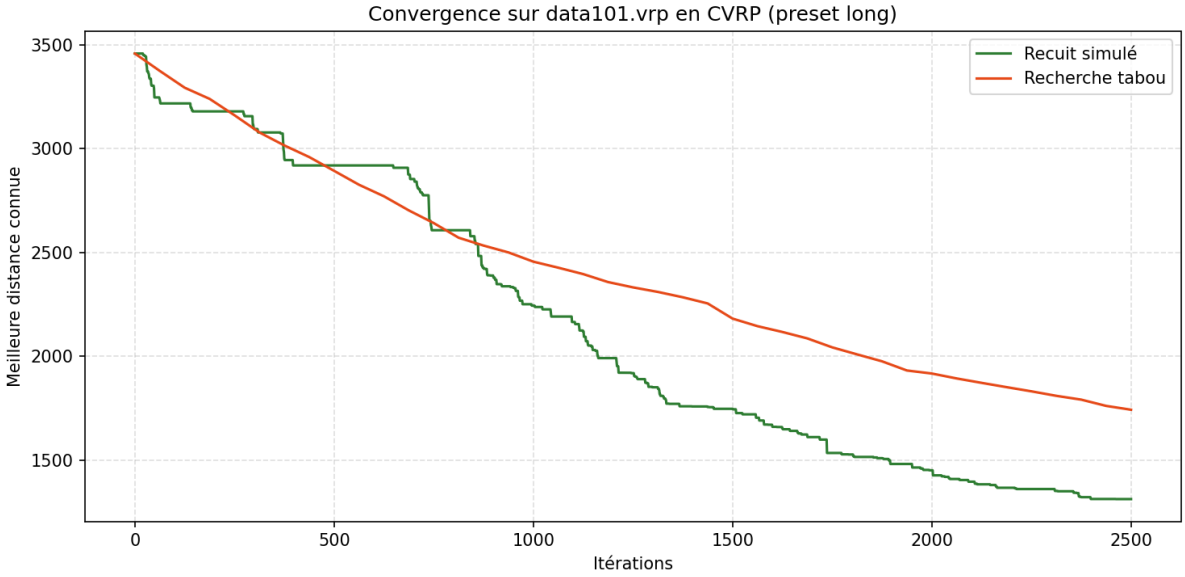

Figure 2: Évolution de la distance (convergence) sur `data101.vrp` en CVRP.

Comme montré sur la figure précédente, le recuit simulé améliore très rapidement la solution dès les premières centaines d'itérations, puis continue à progresser plus finement. La recherche tabou, évaluée sur un nombre d'itérations beaucoup plus faible mais avec une exploration plus structurée à chaque étape, améliore elle aussi la solution initiale, mais converge ici vers un palier plus élevé. Cette figure confirme donc visuellement l'écart de qualité déjà mis en évidence dans les tableaux, tout en montrant que les deux méthodes ne parcourent pas l'espace des solutions de la même manière.

8.17. Courbe de convergence en VRPTW

Une seconde courbe de convergence est ajoutée dans le cadre du vrai problème avec fenêtres de temps, sur l'instance `data1201.vrp`. Cette figure est importante car elle permet d'observer le comportement des deux méthodes lorsque l'espace faisable est fortement restreint par les contraintes temporelles.

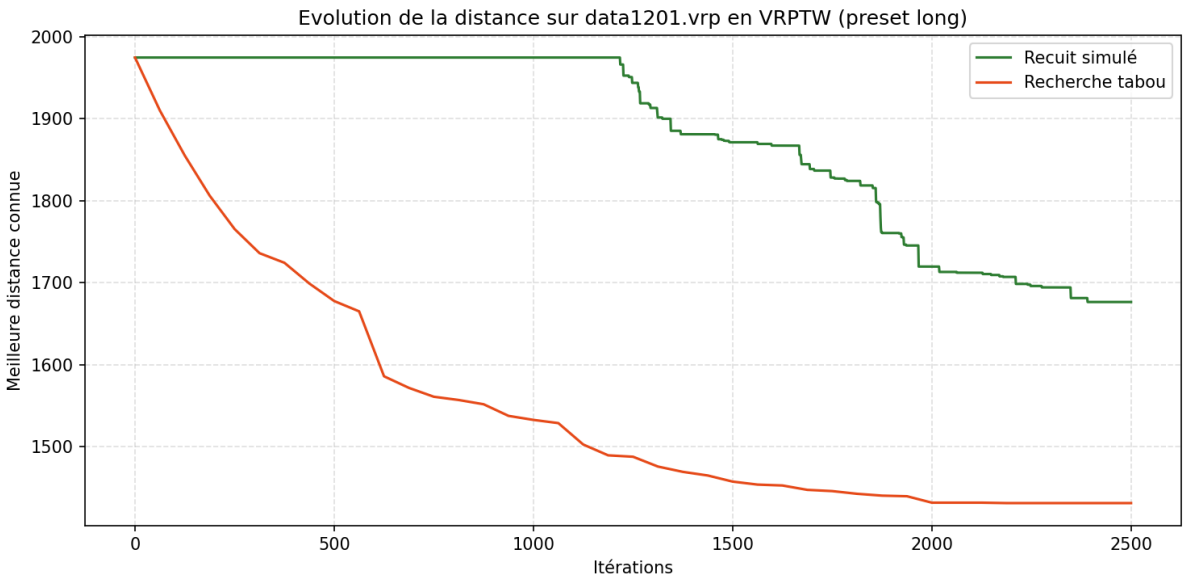

Figure 3: Évolution de la distance (convergence) sur `data1201.vrp` en VRPTW.

Comme montré sur cette figure, le recuit simulé améliore régulièrement la solution initiale, mais la recherche tabou finit ici par descendre plus bas. Cette observation est cohérente avec les résultats moyens de la campagne longue VRPTW : lorsque les contraintes temporelles resserrent fortement l'espace faisable, une exploration plus structurée peut devenir plus avantageuse qu'une exploration légère mais très fréquente.

8.18. Extension du test long à `data102.vrp`

Nous avons ensuite appliqué le même protocole `long` à l'instance `data102.vrp`. La distance moyenne initiale obtenue est 3647.26. Le recuit simulé atteint alors une distance moyenne de 1572.25, tandis que la recherche tabou obtient 1846.33.

Les temps moyens observés sont du même ordre que pour `data101.vrp` : environ 0.56 seconde pour le recuit simulé contre environ 1.33 secondes pour la recherche tabou.

Ce second résultat renforce la tendance observée sur la première instance : dans notre implémentation actuelle et avec les paramètres retenus, le recuit simulé offre un compromis nettement plus favorable entre qualité de solution et temps de calcul que la recherche tabou.

8.19. Résultats sur `data1101.vrp` et `data1102.vrp`

Nous avons ensuite appliqué le même protocole `long` aux instances `data1101.vrp` et `data1102.vrp`, qui présentent une demande totale plus élevée et nécessitent 9 véhicules dans la version sans fenêtres de temps.

Pour `data1101.vrp`, la distance moyenne initiale observée est 4635.30. Le recuit simulé obtient une distance moyenne de 1971.79, tandis que la recherche tabou atteint 2301.35.

Pour `data1102.vrp`, la distance moyenne initiale est 4783.86. Le recuit simulé descend à 1988.77 en moyenne, alors que la recherche tabou obtient 2344.61.

Les temps moyens restent faibles sur ces deux instances : environ 0.76 et 0.74 seconde pour le recuit simulé, contre environ 1.77 et 1.94 secondes pour la recherche tabou.

Ces nouveaux résultats confirment la tendance déjà observée sur `data101.vrp` et `data102.vrp`. Même sur des instances plus chargées, le recuit simulé améliore très fortement les solutions initiales pour un coût de calcul modéré, alors que la recherche tabou demeure beaucoup plus coûteuse tout en restant, dans notre implémentation actuelle, moins performante en qualité de solution finale.

8.20. Résultats sur `data111.vrp` et `data112.vrp`

Le même protocole `long` a ensuite été appliqué aux instances `data111.vrp` et `data112.vrp`. Les résultats observés confirment les mêmes tendances favorables au recuit simulé, même si les valeurs ne coïncident plus exactement avec celles de `data101.vrp` et `data102.vrp` dans notre relance finale.

On observe ici :

- `data111.vrp` : initiale 3498.39, recuit 1531.17, tabou 1806.32 ;
- `data112.vrp` : initiale 3629.29, recuit 1484.82, tabou 1831.06.

Les temps mesurés restent modestes pour les deux méthodes, autour de 0.54 à 0.70 seconde pour le recuit simulé et de 1.44 à 1.62 secondes pour la recherche tabou. Ces deux instances renforcent donc la cohérence de la tendance globale en faveur du recuit dans la version capacitaire.

8.21. Résultats sur `data1201.vrp` et `data1202.vrp`

Nous avons ensuite testé le protocole `long` sur les instances `data1201.vrp` et `data1202.vrp`, qui se distinguent par une capacité beaucoup plus élevée et un nombre de véhicules minimal égal à 2.

Pour `data1201.vrp`, la distance moyenne initiale observée est 4441.38. Le recuit simulé atteint une distance moyenne de 1531.84, tandis que la recherche tabou obtient 1870.48.

Pour `data1202.vrp`, la distance moyenne initiale vaut 4483.64. Le recuit simulé descend à 1594.21 en moyenne, alors que la recherche tabou atteint 1892.68.

Le point notable ici est que les deux méthodes deviennent très rapides : environ 0.54 à 0.62 seconde pour le recuit simulé et environ 1.29 à 1.43 secondes pour la recherche tabou.

Ces résultats renforcent encore l'avantage pratique du recuit simulé dans notre implémentation actuelle : même lorsque la structure des solutions change fortement, il reste à la fois efficace en qualité et très peu coûteux en temps de calcul.

8.22. Résultats sur `data201.vrp` et `data202.vrp`

Enfin, nous avons appliqué le protocole `long` aux instances `data201.vrp` et `data202.vrp`, qui utilisent elles aussi 2 véhicules dans la version sans fenêtres de temps.

Pour `data201.vrp`, la distance moyenne initiale observée est 3432.02. Le recuit simulé obtient une distance moyenne de 1276.75, tandis que la recherche tabou atteint 1614.32.

Pour `data202.vrp`, la distance moyenne initiale vaut 3451.40. Le recuit simulé descend à 1313.29 en moyenne, alors que la recherche tabou obtient 1525.08.

Comme pour `data1201.vrp` et `data1202.vrp`, les temps restent très faibles : environ 0.45 à 0.52 seconde pour le recuit simulé et autour de 1.11 à 1.21 secondes pour la recherche tabou.

8.23. Synthèse de la première campagne longue

À l'issue de cette première campagne longue menée sur l'ensemble des familles d'instances disponibles, une tendance forte se dégage :

- le recuit simulé améliore systématiquement et fortement les solutions initiales ;
- la recherche tabou améliore également nettement les solutions après recalibrage, mais reste globalement en retrait par rapport au recuit simulé dans la version capacitaire ;
- le coût de calcul du recuit simulé reste faible sur toutes les instances testées ;
- le coût de calcul de la recherche tabou reste raisonnable, généralement entre 4 et 17 secondes par bloc de cinq répétitions.

Le tableau suivant résume les valeurs moyennes observées sur l'ensemble de cette première campagne longue.

Instance	k	Init. moy.	RS moy.	Tabou moy.	Temps RS (s)	Temps tabou (s)
data101.vrp	8	3655.35	1305.33	1367.68	2.55	10.45
data102.vrp	8	3537.80	1293.09	1365.31	2.58	11.09
data1101.vrp	9	4718.00	1698.95	1747.80	4.01	16.24
data1102.vrp	9	4675.37	1630.59	1723.34	3.90	16.35
data111.vrp	8	3519.79	1303.22	1350.86	2.23	8.94
data112.vrp	8	3542.86	1278.01	1358.68	2.23	9.05
data1201.vrp	2	4613.08	1348.31	1327.67	1.34	5.27
data1202.vrp	2	4366.31	1310.44	1343.73	1.31	5.51
data201.vrp	2	3580.75	1136.44	1177.15	1.04	4.51
data202.vrp	2	3379.91	1116.25	1149.75	1.22	4.72

Nous observons ici un signal très net : dans la version capacitaire, le recuit simulé reste la meilleure méthode en qualité sur l'ensemble du corpus. Cette différence s'explique par le fait qu'en CVRP l'espace faisable est plus large et se prête bien à une exploration légère mais très fréquente, ce qui constitue précisément l'avantage du recuit simulé dans notre implémentation. La recherche tabou recalibrée devient néanmoins beaucoup plus compétitive qu'auparavant sans coûter significativement plus cher.

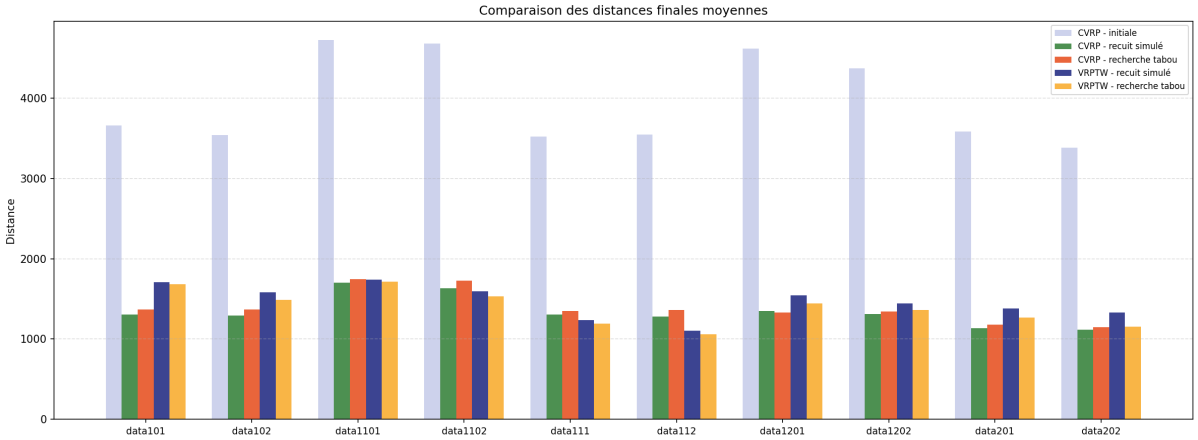

Figure 4: Comparaison des performances en distance finale sur les campagnes longues CVRP et VRPTW.

Comme montré sur cette figure, le recuit simulé domine nettement le corpus CVRP, tandis que la recherche tabou devient plus souvent la meilleure méthode en VRPTW. Cette lecture synthétique confirme que les performances relatives des métaheuristiques dépendent fortement de la structure faisable du problème.

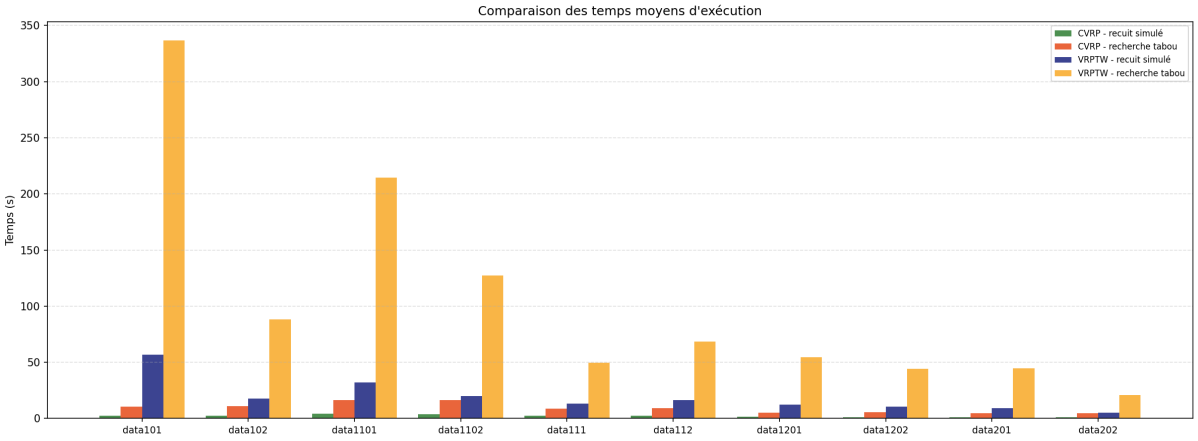

Figure 5: Comparaison des performances en temps d'exécution sur les campagnes longues CVRP et VRPTW.

Comme montré sur cette figure, la recherche tabou reste plus coûteuse en temps que le recuit simulé, surtout en VRPTW. Cela implique qu'une meilleure qualité de solution ne s'obtient pas gratuitement : le compromis qualité/temps reste un élément central de l'analyse.

8.24. Lecture synthétique du corpus et cas marquants

Les campagnes finales `long` couvrent l'ensemble des dix instances du corpus, en CVRP comme en VRPTW. Nous disposons donc bien d'une base expérimentale complète pour dégager des tendances globales ; en revanche, toutes les instances n'ont pas besoin d'un commentaire détaillé dans le corps du rapport. Nous retenons ici trois cas particulièrement informatifs.

`data101.vrp` constitue d'abord le cas VRPTW le plus contraint de notre corpus, avec 21 véhicules nécessaires contre 8 en CVRP. Sur cette instance, la recherche tabou l'emporte avec une distance moyenne de 1683.01 contre 1706.28 pour le recuit simulé (moyenne sur cinq répétitions). L'écart de 23 unités (≈1,3 %) illustre que, même dans un espace faisable très contraint, la mémoire tabou permet d'exploiter plus finement les mouvements disponibles. Le coût en temps reste élevé pour la tabou (≈337 s par répétition contre ≈57 s pour le recuit simulé), ce qui en fait la méthode à privilégier lorsque la qualité prime sur le temps de calcul. Cette instance joue donc le rôle de cas limite, utile pour comprendre le coût réel des contraintes temporelles les plus sévères.

`data1201.vrp` est ensuite un cas charnière, car il illustre très clairement le changement de régime entre CVRP et VRPTW. Sans fenêtres de temps, 2 véhicules suffisent ; la tabou passe devant légèrement (1327.67 contre 1348.31 pour le RS). Avec fenêtres de temps, l'instance passe à 5 véhicules et l'avantage tabou se creuse nettement (1445.97 contre 1544.00). La visualisation des tournées montre concrètement cette fragmentation supplémentaire et rend visible l'effet structurel des contraintes temporelles.

Enfin, `data202.vrp` constitue un cas atypique très instructif en VRPTW. Sur cette instance, le recuit simulé améliore modestement la solution initiale (de 1600.20 à 1514.41 en moyenne), tandis que la recherche tabou descend jusqu'à 1238.85. L'écart est particulièrement marqué

(276 unités), ce qui suggère que certains bassins locaux nécessitent une mémoire explicite pour être quittés efficacement, et qu'une exploration plus structurée avec mémoire tabou peut alors devenir décisive.

Cette lecture par cas ne remplace donc pas l'analyse globale ; elle la complète. Le corpus complet sert à établir les tendances robustes, tandis que quelques instances ciblées permettent d'expliquer pourquoi ces tendances apparaissent.

8.25. Exemple de visualisation de tournées

La figure suivante illustre une visualisation explicite des tournées sur l'instance `data1201.vrp`. Afin de garder une figure lisible, nous représentons d'un côté une solution CVRP obtenue par recuit simulé et, de l'autre, une solution VRPTW obtenue par recherche tabou. Ces deux solutions sont effectivement reconstruites à partir du code final et d'une graine contrôlée.

Cette visualisation met en évidence un point important du projet : sur la même géométrie de clients, l'introduction des fenêtres de temps modifie fortement la structure des tournées et augmente le nombre de véhicules nécessaires. Dans le cas CVRP, la solution reste concentrée autour de 2 tournées, alors que la version VRPTW conduit ici à 5 tournées plus fragmentées.

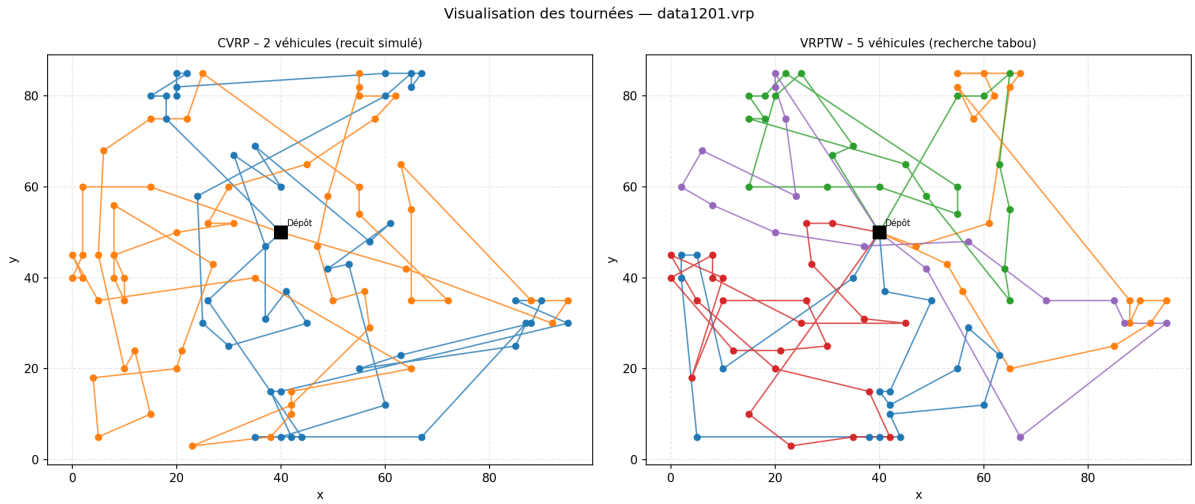

Figure 6: Visualisation comparative des tournées sur `data1201.vrp` : CVRP à gauche, VRPTW à droite.

9. Format des données

Les fichiers d'entrée sont des fichiers texte au format `.vrp`. Une instance contient :

- le nom du fichier ;
- le nombre de dépôts ;
- le nombre de clients ;
- la capacité maximale d'un véhicule ;
- les informations sur le dépôt ;
- les informations sur chaque client.

Exemple de structure :

```
NAME: data101.vrp
COMMENT:
TYPE: vrptw
COORDINATES: cartesian
NB_DEPOTS: 1
NB_CLIENTS: 100
MAX_QUANTITY: 200

DATA_DEPOTS [idName x y readyTime dueTime]:
d1 35 35 0 230

DATA_CLIENTS [idName x y readyTime dueTime demand service]:
c1 41 49 161 171 10 10
...
```

Chaque client est donc décrit par :

- ses coordonnées (x, y) ;
- sa fenêtre de temps $[readyTime, dueTime]$;
- sa demande ;
- sa durée de service.

10. Bonus : résolution exacte sur sous-instances

Le sujet proposait en bonus d'utiliser un package de programmation linéaire afin d'identifier à partir de quelle taille d'instance il devient difficile d'obtenir une solution exacte. Nous avons traité ce point avec OR-Tools, via le solveur linéaire SCIP, sur la version capacitaire du problème.

L'idée retenue est simple : partir d'une instance existante, construire des sous-instances contenant seulement les n premiers clients, puis résoudre exactement chacune de ces sous-instances avec une limite de temps fixée. Nous avons automatisé cette étude dans un module dédié, de façon à pouvoir relancer facilement les tests sur plusieurs tailles.

Pour cette première étude, nous sommes partis de `data101.vrp` et avons testé les tailles 5, 10, 15, 20, 25, 30, 35 et 40 clients, avec une limite de 10 secondes par sous-instance.

Clients	k	Statut	Objectif	Temps (s)
5	1	OPTIMAL	119.48	0.03
10	1	OPTIMAL	173.04	0.01
15	2	OPTIMAL	248.91	0.74
20	2	OPTIMAL	280.13	0.46
25	2	OPTIMAL	335.27	5.08
30	3	FEASIBLE	364.88	10.01
35	3	FEASIBLE	444.35	10.01
40	3	FEASIBLE	561.89	10.01

L'enseignement principal est net : dans notre configuration, le solveur exact reste très efficace jusqu'à 25 clients, mais commence à peiner dès 30 clients, où il atteint déjà la limite de temps sans certifier l'optimalité. Ce seuil ne constitue pas une frontière théorique universelle, mais il

donne un ordre de grandeur concret sur le moment où une approche exacte devient sensiblement plus difficile à exploiter dans ce projet.

11. Limites et perspectives

Le travail réalisé permet de répondre à l'ensemble des points principaux du sujet, mais certaines limites doivent être signalées.

- Dans le cas VRPTW, la génération aléatoire de solutions initiales a été nettement améliorée, mais elle reste moins robuste que dans la version purement capacitaire.
- La recherche tabou recalibrée obtient de très bons résultats sur la campagne finale VRPTW, mais son coût de calcul demeure plus élevé sur quelques instances difficiles, en particulier lorsque le nombre de véhicules imposé par les fenêtres de temps devient important.
- L'étude exacte avec OR-Tools a volontairement été limitée à des sous-instances de petite taille ; elle fournit donc un ordre de grandeur utile, mais pas une frontière générale de difficulté.

Ces limites ouvrent des perspectives naturelles pour une poursuite du projet : renforcer encore l'initialisation aléatoire en VRPTW, affiner les paramètres ou la mémoire tabou sur les cas les plus contraints, et élargir l'étude exacte à d'autres familles de sous-instances.

12. Conclusion

Ce projet aboutit à une chaîne complète de résolution du problème, de la modélisation aux campagnes expérimentales, en passant par la construction de solutions initiales, les voisinages et deux métaheuristiques distinctes.

Trois résultats ressortent nettement. D'abord, sur le corpus CVRP étudié, la borne inférieure capacitaire est atteinte pour toutes les instances et coïncide donc avec le nombre minimal de véhicules observé. Ensuite, l'introduction des fenêtres de temps augmente fortement le nombre de véhicules nécessaires et resserre l'espace des solutions faisables. Enfin, les performances relatives des métaheuristiques dépendent de cette structure : le recuit simulé constitue le meilleur compromis en CVRP (9 instances sur 10), alors que la recherche tabou devient systématiquement meilleure en VRPTW sur la totalité du corpus dans la campagne longue finale.

12.1. Comment nous cherchons l'optimum

Notre stratégie de résolution repose sur une décomposition en trois couches.

Couche 1 — Construction. Nous déterminons d'abord le nombre minimal de véhicules k par une heuristique gloutonne géographique, en incrémentant depuis la borne inférieure capacitaire LB jusqu'à obtenir une solution faisable. En VRPTW, une construction déterministe par insertion prend le relais lorsque la construction aléatoire échoue. Cette solution initiale fournit un point de départ réaliste mais sous-optimal.

Couche 2 — Voisinage local. À partir de la solution initiale, nous explorons un voisinage défini par cinq opérateurs : *relocate* (déplacement d'un client vers une autre position ou tournée), *swap* (échange de deux clients quelle que soit leur tournée), *2-opt* intra-route (inversion d'un segment dans une même tournée), *Or-opt(2)* (déplacement d'une paire de clients consécutifs) et *2-opt** (échange des suffixes de deux tournées distinctes). Chaque candidat est validé par

le module `evaluate.py` avant d'être proposé à la métaheuristique, garantissant que seules des solutions faisables sont examinées.

Couche 3 — Métaheuristiques. Les deux métaheuristiques pilotent l'exploration à l'échelle globale :

- le recuit simulé accepte des mouvements dégradants avec probabilité $\exp(-\Delta/T)$, ce qui permet d'échapper aux minima locaux ; la température T décroît géométriquement selon le facteur de refroidissement $\alpha = 0{,}997$, réduisant progressivement l'exploration au profit de l'exploitation ;
- la recherche tabou interdit les solutions récemment visitées (mémoire FIFO de taille *tenure* = 20) et sélectionne toujours le meilleur voisin non tabou, avec un critère d'aspiration pour ne pas bloquer une amélioration globale.

Ces conclusions reposent sur l'ensemble du corpus final et non sur quelques exemples isolés ; les études de cas détaillées servent seulement à interpréter les comportements les plus marquants.

12.2. Améliorations envisageables

Plusieurs directions permettraient encore d'améliorer la qualité des résultats.

Heuristique de construction plus performante. L'algorithme Solomon I1, qui insère les clients un par un en maximisant un critère combinant économie de distance et urgence temporelle, fournirait des solutions initiales de meilleure qualité que notre greedy géographique, notamment en VRPTW fortement contraint. Une meilleure initialisation réduirait également la dépendance aux répétitions multiples.

Refroidissement adaptatif. Plutôt qu'un facteur de refroidissement géométrique fixe ($\alpha = 0{,}997$), un refroidissement adaptatif qui ajuste la température en fonction du taux d'acceptation des dernières itérations permettrait de mieux calibrer l'exploration sans réglage manuel des paramètres.

Séquence d'opérateurs guidée. Plutôt qu'un tirage uniforme parmi les cinq opérateurs, une stratégie adaptative qui favorise les opérateurs ayant récemment produit des améliorations (bandit multi-bras) exploiterait plus efficacement le budget de calcul disponible.

12.3. Critique du protocole expérimental

Diversité des solutions initiales en VRPTW. En VRPTW fortement contraint, la construction aléatoire échoue presque systématiquement (taux d'échec supérieur à 97 % sur `data101.vrp`). Notre filet de sécurité — la construction déterministe — retourne alors toujours la même solution initiale pour toutes les répétitions. Conséquence directe : les répétitions démarrent du même point et la variabilité observée ne reflète que les fluctuations stochastiques intra-run, et non une vraie diversité inter-runs. Une initialisation basée sur plusieurs heuristiques déterministes différentes (greedy géographique, greedy temporel, Solomon I1) corrigerait ce biais.

Nombre de répétitions et significativité statistique. Notre préréglage `long` utilise cinq répétitions. L'erreur standard sur la moyenne est $\sigma/\sqrt{n}$: avec $n = 5$ on obtient $\sigma/2{,}24$, contre $\sigma/3{,}16$ avec $n = 10$. Augmenter à dix répétitions diviserait l'incertitude par 1,41 supplémentaire, au prix d'un facteur $\times 2$ sur le temps de calcul. C'est un compromis raisonnable pour une étude définitive.

Mémoire tabou par signature globale. Nous encodons chaque solution par sa signature complète (tuple trié des routes), qui est une représentation grossière : deux solutions très dif-

férentes topologiquement peuvent produire la même signature si elles visitent les mêmes clients dans des ordres différents. Une mémoire tabou basée sur les mouvements eux-mêmes (paire de clients concernés, type d'opérateur) serait plus précise et permettrait un tenure plus long sans sur-interdire de bons candidats.